CHRONOS :: LEARNING MANUAL

How to learn everything you need to build your own shell, by yourself

You already have a working Quickshell shell, a niri fork and a NixOS flake. That is a big head start: you are not learning in the abstract, you have a real machine (the Surface) and a real roadmap (95 items). This manual turns that roadmap into a learning path. It tells you **what to learn**, **in what order**, **how to practise it** and **how to know you have learned it**.

Time estimates assume about 6 hours a week. They are rough guesses, not promises. Yours will vary, and that is fine: the order matters more than the speed.

0. THE THREE RULES

- **Build small, then build real.** Every concept gets a tiny throwaway project (30 minutes to 2 hours) before you touch Chronos. Chronos is the final exam, not the classroom.
- **Read real code.** Reading other people's shells, niri and Smithay teaches patterns no tutorial covers. Read first, then write your own version. Do not paste what you cannot explain.
- **Break things on purpose.** Change a value, delete a line, predict what happens, then check. Prediction followed by surprise is where most learning happens.

The learning loop for every topic

Step	Share of time	What it looks like
READ	20%	One chapter or doc page. Stop as soon as you can try something.
DO	60%	Type it yourself (no copy-paste), change it, break it, fix it.
EXPLAIN	20%	Write 5 lines in your notes as if teaching a friend. If you cannot, you have not learned it yet.

1. THE MAP: WHAT CHRONOS IS MADE OF

Chronos is a stack. Each layer has its own language and its own way of failing. You can learn them one at a time, and the top layers are much easier than the bottom ones. That is why the path below starts at the top (the shell) and works down to the compositor.

Layer	Technology	Roadmap items	Difficulty
Terminal and tools	zsh, ssh, git, systemd, journalctl	everything	easy
Shell UI	QML + Quickshell (JavaScript-like)	1.2x, 1.3x, 2.2x, 3.0x, 4.xx	easy to medium
Packaging	Nix, flakes, modules, home-manager	1.04-1.06, 1.36, 3.11, 5.01	medium (steep start)
Wayland concepts	protocols: layer-shell, session-lock, text-input	1.01, 1.12	medium (concepts)
Input stack	evdev, libinput, touch, pen, sensors	1.10, 1.11, 1.13, 2.40	medium to hard
Compositor	Rust, niri, Smithay	1.10, 1.12, 1.15, 2.01-2.11	hard
Graphics	GLSL shaders, dithering algorithms	3.02, 3.03, 3.04	medium
Design and art	type, palette, pixel art, ASCII	3.0x, 4.01	different skill, not harder
Quality	tests, CI, docs, releases	5.0x	easy to medium

Look at the dependency map in the roadmap again. Almost every chain ends in a shell item, and only a few start in the compositor (palm rejection, rotation, the text-input hook). So you can have a very complete daily driver before you ever write a line of Rust. Do that first: it keeps you motivated and it makes the compositor work easier, because you will know exactly what the shell needs from it.

2. THE LEARNING PATH (9 STAGES)

Stages 1 to 4 can overlap. Stages 5 to 7 are the deep end and should come in order. Stages 8 and 9 can run alongside anything once you start Tier 3.

STAGE 1 :: FOUNDATIONS	2-4 weeks
GOAL	Be comfortable living in a terminal and never lose work to git.
LEARN	- Shell basics: paths, pipes, redirection, `find`, `rg`, `sed`, `jq`, permissions, environment variables. - Processes and services: `systemctl --user`, unit files, `journalctl` (your first debugging tool). - SSH, `scp`, keys, so you can drive the Surface from the desktop. - Git: commit, branch, `diff`, `stash`, rebase, `bisect`, reading history. You will need rebase for niri (item 5.05).
PRACTICE	- Write a systemd user service that runs a script every minute and logs to the journal. - Use `git bisect` on your own repo to find the commit that broke something (break it first). - Pipe `niri msg -j workspaces` into `jq` and print only the focused Timeline.
CHECKPOINT	Explain, step by step, what happens from `systemctl --user restart chronos-terminal` until a panel appears.
UNLOCKS	Everything. Most of your past debugging rounds were really journal-reading problems.

STAGE 2 :: PROGRAMMING CORE	3-6 weeks
GOAL	Think in variables, functions, objects, events and data. QML uses JavaScript, so learn that flavour.
LEARN	- Variables, types, functions, objects, arrays, `map`/`filter`, JSON parse and stringify. - Events and callbacks: 'when X happens, run Y'. This is how all UI code works. - Reading error messages calmly: file, line, what it expected, what it got. - Bash/zsh scripting for glue: variables, `if`, loops, functions, exit codes.
PRACTICE	- Write 5 small scripts: battery alert, wallpaper rotator, clipboard logger, window-count printer, 'which app uses most RAM'. - Do 10 easy exercises on a site such as Exercism (JavaScript track).
CHECKPOINT	You can read a 50-line function you did not write and say what it does and where it could fail.
UNLOCKS	The logic inside every service in `shell/services/`.

STAGE 3 :: QML AND QUICKSHELL	6-10 weeks
GOAL	Own the shell layer: you can build any panel, from a blank file, without help.
LEARN	- QML core: `Item`, `Rectangle`, `Text`, anchors, `Row`/`Column`/`Grid`, property bindings (the big idea: values that update themselves). - Animation: `Behavior`, `NumberAnimation`, `states` and `transitions`, `Timer` (your stepped print-in effect is built from these). - Models and views: `Repeater`, `ListView`, `ListModel`, delegates. `Loader` for lazy panels. - Touch input: `TapHandler`, `DragHandler`, `PinchHandler`, `MultiPointTouchArea`, how gestures are claimed. - Quickshell: `PanelWindow` (layer-shell), `Variants` per screen, singletons, `Process` + `SplitParser`, `FileView`, IPC handlers, and the built-in services (Pipewire, UPower, Mpris, notifications, system tray). - Reading Quickshell docs and the C++ types behind them when the docs are thin.
PRACTICE	- The ladder in section 4, items 1 to 10, in order. Each one adds exactly one new idea. - Rebuild one of your existing panels (for example the Index) from an empty file, then diff against yours. Note every difference. - Read two or three public Quickshell configurations (for example Caelestia or end-4's dotfiles) and list five patterns you want to steal. Steal the idea, not the code.

CHECKPOINT	You can add a new Controls row with a slider and a live value in under an hour, and explain every property you used.
UNLOCKS	Tier 1: 1.20-1.24, 1.30-1.34, 1.38. Tier 2: 2.04-2.05, 2.20-2.30. Tier 3: 3.05-3.07, 3.16. Tier 4: all of it.

STAGE 4 :: NIX IN DEPTH

3-5 weeks (parallel)

GOAL	Stop copying Nix snippets. Write modules, options and packages yourself.
LEARN	- The language: attribute sets, functions, `let ... in`, `inherit`, `with`, laziness. Practise in `nix repl`. - Derivations and the store: why paths look like `/nix/store/hash-name` and why that makes rollbacks free. - The module system: `options`, `config`, `mkOption`, `mkIf`, `mkMerge`, `imports`. Your `defaultMode` option is a perfect small example. - Flakes: inputs, outputs, the lock file, `nix flake update`, `checks`, `devShells`. - Packaging: `rustPlatform.buildRustPackage`, `stdenv.mkDerivation`, overlays. home-manager and systemd user units from Nix.
PRACTICE	- Add a second option to `chronos-terminal.nix` (for example idle timeout) and export it as an env var. - Create a `devShell` with qmllint, jq and the tools from section 6 so any machine can build and debug Chronos. - Package a 20-line Rust 'hello' program as a flake package, then add it to your system.
CHECKPOINT	You can explain why 'never deploy a dirty tree' is a rule, and what each step of your deploy does.
UNLOCKS	1.04, 1.06, 1.36, 1.40, 3.11, 3.13, 5.01, 5.08, 5.09. Also makes every future rebuild less scary.

STAGE 5 :: WAYLAND AND THE INPUT STACK

4-6 weeks

GOAL	Understand what a compositor, a shell and an app say to each other, and how a finger on glass becomes an event.
LEARN	- Wayland model: compositor, client, surface, buffer, seat, output. Why there is no global 'draw anywhere' like in X11. - Protocols your project depends on: xdg-shell, wlr-layer-shell (panels), ext-session-lock (Clearance), text-input and input-method (OSK auto-show), virtual-keyboard (what `wtype` uses), foreign-toplevel (Dock, Dossier list). - Input chain: kernel evdev -> libinput -> compositor -> client. Touch slots, gesture events, palm detection, tablet-mode switch, pen events. - Sensors: `iio-sensor-proxy` for rotation. Surface-specific: how the touch driver and the Type Cover attach/detach report to the kernel (verify on your own device with the tools below).
PRACTICE	- Run `wayland-info` and read which protocols niri offers. Look each unknown one up in a protocol browser. - Run `libinput debug-events` over SSH while doing 3f and 4f swipes on the Surface. Compare with your gesture-core logic. - Run an app with `WAYLAND_DEBUG=1` and find the exact line where it asks for a surface and a buffer. - Optional: write a 100-line Wayland client (any language) that opens a coloured window.
CHECKPOINT	You can explain, with arrows, how an edge swipe on the left becomes a moving Index panel, naming every component involved.
UNLOCKS	1.01, 1.10-1.15, 2.40. This stage is the bridge to compositor work.

STAGE 6 :: RUST

8-12 weeks

GOAL	Be able to read and change niri's code without fear of the borrow checker.

LEARN	- Order: The Rust Book chapters 1-10, then 13, 15, 16. Do Rustlings alongside. - Ownership and borrowing, `struct`/`enum`/`match`, `Option`/`Result` and the ? operator, traits, generics, iterators, closures. - Cargo, crates, modules, `serde` for JSON, `anyhow` for errors, channels, and the idea of an event loop (niri uses calloop). - Tests: `#[test]`, `cargo test`. You already have gesture-core tests, so read them as Rust examples.
PRACTICE	- Section 4, items 11 and 12: an event-stream printer and a tiny socket daemon. - Re-implement your gesture-core threshold logic in 100 lines of your own Rust, then compare with the fork. - Fix three small warnings from `cargo clippy` in the fork, one commit each.
CHECKPOINT	You can explain, without notes, why this fails to compile and how to fix it: borrowing a value after moving it into a function.
UNLOCKS	Everything in the niri layer. Do not skip this stage: guessing at Rust is the slowest path there is.

STAGE 7 :: COMPOSITOR INTERNALS (niri + Smithay)

3-6+ months, ongoing

GOAL	Make safe, small, rebase-friendly patches to the fork.
LEARN	- Orientation: read niri's wiki and architecture notes, then trace one feature end to end (for example how a keybind becomes a workspace switch). - niri's layout model: outputs, workspaces, scrolling columns, windows. Where focus lives. How IPC events are produced (the `niri-ipc` crate). - Input handling in the fork: where libinput gestures arrive, where yours are claimed, how thresholds are stored. - Smithay basics: its examples (a small compositor called smallvil is a good first read), Wayland object handling, rendering elements, GLES shaders. - Keeping patches small so you can rebase on upstream (item 5.05).
PRACTICE	- Patch ladder, in this order: (1) add a log line on 4f events, (2) fix 1.15, (3) make thresholds live (1.14), (4) palm rejection (1.10), (5) rotation (1.11), (6) text-input hook (1.12), (7) floating/tabs (2.01, 2.03). - For each patch: write the 3-line goal first, change as little as possible, add a test where the code allows it, record the lesson in your bug diary. - Optional detour for the basics: read a minimal compositor in C (tinywl in wlroots) to see the whole loop in 1000 lines.
CHECKPOINT	You merged an upstream niri update into the fork and your gestures still work.
UNLOCKS	1.10-1.15, 2.01-2.03, 2.06-2.11, 2.40, 3.08, 5.05.

STAGE 8 :: GRAPHICS AND SHADERS

4-8 weeks (parallel, from Tier 3)

GOAL	Build the CRT look with a known, measured cost.
LEARN	- GLSL basics: coordinates, uniforms, `mix`, `step`, `smoothstep`, noise. Build effects one at a time. - CRT toolbox: scanlines, bloom/glow (blur + add), grain, vignette, curvature, chromatic aberration, flicker. - Dithering: ordered (Bayer matrix) and error diffusion (Floyd-Steinberg). Know why 1-bit Bayer gives your 'phosphor' look. - QML `ShaderEffect` (shaders must be precompiled to .qsb with Qt's `qsb` tool) and when to use niri's renderer instead. - Measuring: GPU use, frame time, battery draw (`intel_gpu_top`, `powertop`). This answers your open decision about the GPU budget.
PRACTICE	- Recreate each effect in a browser shader playground first (fast feedback), then port to QML. - Write a Python or ImageMagick script that converts a photo to a 1-bit 8x8 Bayer image (item 3.04). - Measure idle power with and without each effect on the Surface and write the numbers in your notes.
CHECKPOINT	You can show a table: effect, GPU %, watts, keep/off by default.
UNLOCKS	3.02, 3.03, 3.04, 3.07, 4.03, 4.04.

STAGE 9 :: DESIGN, ART AND CRAFT	ongoing
GOAL	Make it look intentional and keep it maintainable.
LEARN	- Design: grid and spacing, type scale, one-hue palettes, contrast (aim for readable text in every hue), restraint. Study your own `docs/style-reference/` and write down its rules as tokens. - Pixel and ASCII art: small, hand-drawn glyphs beat generated ones. Tools: a pixel editor (Aseprite or LibreSprite), GIMP, ImageMagick. - Quality: `qmllint`, `nix flake check`, GitHub Actions, mock-IPC QML tests, writing docs, changelogs, tags and NixOS rollbacks.
PRACTICE	- Write the tokens file (3.01) by hand before building the Settings Dossier. - Draw 10 new ASCII icons (3.05) in a single sitting, then keep the best five. - Set up CI that builds, lints and tests on every push (5.01). Break a test on purpose and watch it fail.
CHECKPOINT	A stranger could read THEME.md and add a new hue without asking you anything.
UNLOCKS	3.01, 3.05, 3.06, 3.09, 3.10, 3.15, 3.16, 5.01-5.08.

3. YOUR ROADMAP, MAPPED TO STAGES

This follows the 'Next up' list from the roadmap. The right-hand column is what you need to understand *before* that step, so nothing ambushes you.

#	Roadmap step	Stages needed	Understand first
1	Test 4f quick-switch and Archive event (1.15)	1, 5	How to read `libinput debug-events` and the event-stream. Observe only; do not patch yet.
2	Retire dynisland, safe-mode fallback (1.40, 1.07)	3, 4	Nix: removing a package and a unit. QML: `Loader` and error handling for the fallback panel.
3	Palm rejection, dead zones, live gesture settings (1.10, 1.14)	5, 6, 7	libinput palm and touch model, Rust basics, niri's gesture code path. Do 1.14 first: it is the gentler patch.
4	Rotation (1.11)	5, 6, 7	iio-sensor-proxy, output transforms in niri, how touch coordinates follow a rotated output.
5	OSK auto-show, Type Cover auto mode (1.12, 1.13)	5, 7	text-input protocol, how niri would emit an event to the shell, tablet-mode switch.
6	Clearance, idle chain, power menu (1.01-1.03)	3, 5	ext-session-lock, idle-notify, logind actions. Mostly shell work.
7	Wifi, Bluetooth, audio pickers (1.20-1.22)	3	`Process`/D-Bus talk to NetworkManager and BlueZ, Pipewire service, `ListView` with actions.
8	Capture and Ledger (1.30, 1.31)	3	Running external tools from QML, clipboard watching, file I/O.
9	Window rules, Dock v2 (1.37, 1.38)	3, 4, 7	niri window rules syntax, app-ids, foreign-toplevel data, persistence.
10	Theme tokens, then CRT effects (3.01, 3.02)	8, 9	JSON-driven theming, ShaderEffect, measuring cost on the Surface.

Suggested adjustment: steps 6 to 8 are all shell work. If compositor work feels too heavy at first, do them *before* steps 3 to 5 and come back to Rust with more confidence. The roadmap order is a priority order, not a law.

4. THE PROJECT LADDER

Seventeen small projects, each adding one idea. Do them in a scratch repo (not inside Chronos), then port the good ones in.

#	Project	New idea	Roadmap link
1	Clock in a top bar (`PanelWindow`)	layer-shell, anchors, Timer	1.33
2	Battery bar with the UPower service	services, property bindings	1.33, 2.22
3	Volume slider bound to Pipewire, with auto-hiding ASCII bar	two-way binding, animation	1.23
4	Now-playing row from MPRIS with play/pause	Mpris service, actions	1.24, 2.26
5	Notification server and popup stack	notification service, ListView	1.32
6	Wifi list: parse `nmcli` output and join a network	Process, parsing, state	1.20
7	Process list with a kill button	Process, models, confirm Dossier	2.24
8	Clipboard history stored in a JSON file	FileView, persistence	1.31
9	Screenshot tool with region select	external tools, UX flow	1.30
10	CPU/RAM waveform from `/proc` on a Canvas	Canvas, sampling, timers	2.23
11	Rust: pretty-print `niri msg -j event-stream`	serde_json, stdin lines	1.15

#	Project	New idea	Roadmap link
12	Rust: tiny daemon on a unix socket	sockets, event loop idea	IPC design
13	In the fork: add a log line and one new IPC event	build, run, read the code	1.15
14	Add a gesture-core test for a new threshold	tests, edge cases	1.14
15	Scanlines + vignette shader, browser first, then QML	GLSL, qsb	3.02, 3.03
16	Bayer 8x8 dither script for images	algorithms, assets	3.04
17	New Nix option plus a flake check	module system, checks	5.01

5. HOW TO LEARN WITH (NOT FROM) AN AI

You have been building with Claude's help. That works for shipping features, but it can hide gaps: you get working code without the mental model. A few habits keep the speed and add the learning.

- **Concept first, code second.** Ask 'explain how X works in 10 lines, no code yet', then 'now show the smallest example'.
- **Write the first draft yourself.** Even a rough one. Then ask for a review: 'what is wrong, what would you question, do not rewrite it'.
- **Annotate every patch.** Ask for a patch with a comment on every changed line, or ask 'explain this diff line by line' before you apply it.
- **Predict before you run.** Write 'I expect X' in your notes, run it, compare.
- **Be quizzed.** End each study session with 'ask me 5 questions about what we just did, one at a time, and correct me'.
- **Ask for the debugging path, not only the fix.** 'What would you check first, and why?' Your own lesson applies: read the journal first.
- **Verify against the docs.** AI can be wrong about versions and APIs. When something surprising appears, check the official docs or the source.

Two files worth keeping in the repo

- `docs/LEARNING.md`: one short entry per concept (what it is, one example, one gotcha). Five lines each.
- `docs/BUGS.md`: the bug diary. Columns: symptom, real cause, how I found it, how I will find it faster next time. Your singleton-with-braces-in-the-header-comment bug is a perfect first entry.

A weekly rhythm that works

Block	Time	What
Learn	2 h	One stage topic: read a little, type a lot.
Build	3 h	One ladder project or one roadmap item.
Review	1 h	Notes, bug diary, commit cleanly, deploy if it is clean, pick next week's target.

6. DEBUGGING TOOLBOX (CHEAT SHEET)

```
# what is the shell saying right now?
journalctl --user -u chronos-terminal --since "-30s" --no-pager

# what does niri see? (events as JSON lines)
niri msg -j event-stream
niri msg -j workspaces | jq '.[] | select(.is_focused)'

# raw input: gestures, touches, switches (run over ssh, needs permission)
sudo libinput debug-events
sudo libinput record > touch.yml      # replayable recording for bug reports

# Wayland
wayland-info                          # which protocols does the compositor offer
WAYLAND_DEBUG=1 some-app 2> log.txt   # every message between app and compositor
wev                                   # keyboard/pointer events of a window

# QML / Nix
qmllint shell/**/*.qml
nix repl                              # try Nix expressions interactively
nix flake check
nix store diff-closures /run/booted-system /run/current-system

# Rust (in the fork)
cargo check && cargo clippy && cargo test
RUST_LOG=debug niri                   # more logging (see niri docs for its exact switches)

# GPU / power on the Surface
sudo intel_gpu_top
sudo powertop
```

Method for any bug: (1) reproduce it, (2) read the logs, (3) shrink it to the smallest case, (4) change one thing at a time, (5) write down the cause. Never apply two fixes at once.

7. RESOURCES

Names and sites are the ones I know well. Websites move, so search the name if a link has changed, and check that the page matches the version you run.

Topic	Resource	Use it for
Linux and shell	The Linux Command Line (William Shotts, free online)	Stage 1 backbone
Git	Pro Git (git-scm.com/book)	branching, rebase, bisect
QML	Qt Quick documentation (doc.qt.io) and the QML Book	types, handlers, animation
Quickshell	quickshell.org documentation, plus public Quickshell configs	services, PanelWindow, IPC; patterns
Nix	nix.dev, Nix Pills, the NixOS and Flakes Book, search.nixos.org	language, modules, flakes, option lookup
Wayland	The Wayland Book (wayland-book.com), wayland.app protocol browser	concepts, protocol reference
Input	libinput documentation (wayland.freedesktop.org/libinput)	gestures, palm detection, tablet mode
Rust	The Rust Book, Rustlings, Rust by Example (all at rust-lang.org)	Stage 6, in that order
Compositor	niri wiki and source (github.com/YaLTeR/niri), Smithay and its examples	Stage 7
Shaders	The Book of Shaders, Shadertoy, learnopengl.com	GLSL and effect experiments
Dithering	Obra Dinn devlog on dithering; Bayer and Floyd-Steinberg articles	1-bit look, item 3.04
Art	Aseprite or LibreSprite, GIMP, ImageMagick	icons, assets, batch dithering

8. A 12-MONTH PLAN (AT ~6 H/WEEK)

Month	Focus	Chronos output
1	Stages 1 and 2. Observe and test.	1.15 tested and documented; LEARNING.md and BUGS.md started.
2-3	Stage 3, ladder 1-6.	Wifi picker (1.20), OSD (1.23), MPRIS (1.24).
3-4	Stage 4 and ladder 7-10.	Retire dynisland (1.40), safe-mode fallback (1.07), Telemetry (2.23).
4-5	Stage 3 depth.	Bluetooth, audio, Capture, Ledger (1.21, 1.22, 1.30, 1.31).
5-7	Stage 6 and ladder 11-12; Stage 5 reading.	Your own event-stream tool; gesture logic re-implemented.
7-9	Stage 7, patches 1-4.	1.15 fix, live thresholds (1.14), palm rejection (1.10).
9-10	Stage 7, patches 5-6.	Rotation (1.11), text-input hook and OSK auto-show (1.12).
10-11	Stages 8 and 9.	Theme tokens (3.01), measured CRT effects (3.02), dither assets (3.04).
12	Quality and review.	CI (5.01), docs (5.06), first tagged release (5.08), retrospective.

Do not treat this as a contract. If a month goes badly, repeat it. If a stage clicks fast, skip ahead. The checkpoints in each stage are the real progress meter, not the calendar.

9. COMMON TRAPS

- **Starting with the compositor.** It is the hardest layer and the least forgiving. Earn it with the shell and Rust first.
- **Rewriting what works.** A rewrite feels productive and often is not. Fix, measure, then decide.
- **Tutorial hell.** If you have read for an hour and typed nothing, stop reading.
- **Big patches to the fork.** They make rebasing on upstream painful. Keep them small and separate.
- **Debugging blind.** Logs first, theory second. Your own lessons file already says this.
- **Testing only on the desktop.** Touch, rotation and the Type Cover only exist on the Surface. Mark those items (T) and plan device time.
- **Perfectionism in Tier 3 and 4.** The Backrooms can wait; a stable daily driver cannot.

Final checkpoint: when you can explain the whole path of a gesture, from finger to pixel, in your own words and in your own code, you have built your own shell.